

DatabaseGenerated Attribute in Entity Framework Core

In this article, I will discuss **DatabaseGenerated Data Annotation Attribute in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing the [MaxLength and MinLength Attribute in Entity Framework Core](#) with Examples.

What is the DatabaseGenerated Attribute in Entity Framework Core?

The DatabaseGenerated attribute in Entity Framework Core specifies how the value of a particular property is generated in the database. It informs EF Core whether the value of a property is generated by the database during insert or update operations or provided by the application. It can be applied to primary, non-primary, and computed keys. It provides control over whether a property's value is:

- Generated by the database upon insertion
- Computed by the database upon insertion or update
- Not generated by the database (i.e., values are provided by the application)

DatabaseGenerated Attribute Class Definition:

If you go to the definition of the DatabaseGenerated Attribute class, you will see the following signature. It is part of the System.ComponentModel.DataAnnotations.Schema namespace. This class has one constructor and one property.

```
namespace System.ComponentModel.DataAnnotations.Schema
{
    ... public class DatabaseGeneratedAttribute : Attribute
    {
        ... public DatabaseGeneratedAttribute(DatabaseGeneratedOption databaseGeneratedOption) ...
        ... public DatabaseGeneratedOption DatabaseGeneratedOption { get; }
    }
}
```

As you can see in the above image, the DatabaseGenerated class constructor takes one parameter of type DatabaseGeneratedOption.

DatabaseGeneratedOption Enum Definition:

If you go to the definition of DatabaseGeneratedOption, you will see that it is an Enum with three values, as shown in the image below.

```

namespace System.ComponentModel.DataAnnotations.Schema
{
    //
    // Summary:
    //     Represents the pattern used to generate values for a property in the database.
    public enum DatabaseGeneratedOption
    {
        //
        // Summary:
        //     The database does not generate values.
        None = 0,
        //
        // Summary:
        //     The database generates a value when a row is inserted.
        Identity = 1,
        //
        // Summary:
        //     The database generates a value when a row is inserted or updated.
        Computed = 2
    }
}

```

DatabaseGenerated Enum Values:

- **None:** The database does not generate the value. The application must provide the value.
- **Identity:** The database generates the value when inserting. It is typically used for auto-incrementing columns like primary keys.
- **Computed:** The database generates the value on insert and update. This is useful for columns whose values are calculated based on other columns.

When and How to Use None Option with the Primary Key Column

The None option is useful when we want the application to generate the primary key value instead of the database. It is useful when using GUIDs or other custom key generation strategies. The database does not need to generate the value automatically.

To use this option, apply the **[DatabaseGenerated(DatabaseGeneratedOption.None)]** attribute to the primary key property. Please ensure the application assigns a value to the primary key before saving the entity.

Example: Using GUIDs as primary keys where the application generates the GUID instead of the database. Scenarios where we need the primary key values from external systems.

When and How to Use Identity Option with the Non-Primary Key Column

The Identity Option is useful when a non-primary key column needs to have a unique sequential value generated by the database upon insertion, such as a column that auto-increments but isn't the primary key.

To use this option, apply the **[DatabaseGenerated(DatabaseGeneratedOption.Identity)]** attribute to the property. The database will generate a value for this column when a new record is inserted.

Example: A "SerialNumber" column that auto-increments independently of the primary key.

When and How to Use Computed Option with Static and Dynamic Data

The Computed Option is useful for columns whose values are calculated by the database based on other columns. They can be used for both static data (calculated once) and dynamic data (calculated on insert and update).

To use the Computed Option, apply the **[DatabaseGenerated(DatabaseGeneratedOption.Computed)]** attribute to the property. We also need to configure how the computed value will be calculated within the DbContext class.

Example:

- A TotalPrice column that multiplies "Quantity" by "UnitPrice".
- A CreatedOn column should be set based on the current date and time.

Real-time Example to Understand DatabaseGenerated Attribute in EF Core:

We will build an E-Commerce Console Application using EF Core's Code First approach. The application will manage Products, Customers, Orders, and OrderItems. Each entity will demonstrate different uses of the DatabaseGenerated attribute options (Identity, None, Computed) to meet specific business needs in a .NET Console application.

Define the Entities:

We will define the Product, Customer, Order, and OrderItem entities to explain the use of Identity, None, and Computed options. Create a folder named Entities, where we will create all our entities and the DbContext class.

Product Entity

Create a class file named **Product.cs** within the Entities folder and then copy and paste the following code.

```
using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [Index(nameof(SKU), IsUnique = true)]
    [Index(nameof(SerialNumber), IsUnique = true)]
    public class Product
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.None)] // Application must
provide the GUID.
        public Guid ProductId { get; set; } // Non-Identity Primary Key
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int SerialNumber { get; set; } // Non-Primary Key, Identity
        public string Name { get; set; }
        public string Category { get; set; }
        [Column(TypeName = "decimal(10,2)")]
        public decimal Price { get; set; }
        public int Quantity { get; set; }
        //PROD-{Category}-{SerialNumber}
        [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
        public string SKU { get; set; } // Unique, Non-Primary Key
    }
}
```

```

        public DateTime CreatedOn { get; set; } // Default value, Current
Date
        public string CreatedBy { get; set; } // Fixed value System
    }
}

```

Explanation of Product Entity:

ProductId (DatabaseGeneratedOption.None):

The ProductId is a GUID that must be explicitly generated by the application rather than being auto-generated by the database. Using DatabaseGeneratedOption.None allows us to maintain control over the ProductId and generate it based on our application logic. A GUID ensures global uniqueness, which is often important when products are created and shared across distributed systems or integrated with external APIs.

SerialNumber (DatabaseGeneratedOption.Identity):

The SerialNumber is an auto-generated unique value for each product. It's an integer that is sequentially generated by the database. This is useful for providing human-readable sequential IDs that help with tracking and ordering, without using them as a primary key. Since the primary key (ProductId) is a GUID, having a sequential number (SerialNumber) as an additional identifier can help in inventory and auditing purposes. It acts like a serial number printed on physical products.

SKU (DatabaseGeneratedOption.Computed):

The SKU is computed using a specific formula that concatenates the product's Category and SerialNumber (PROD-{Category}-{SerialNumber}). This value is generated by the database when the record is inserted. This kind of computed column is used when we want to generate a unique SKU based on a fixed formula. The value is computed and stored in the database, allowing the SKU to be unique and consistent based on other attributes.

CreatedOn and CreatedBy:

CreatedOn uses the current date as a default value (GETDATE()). This ensures the timestamp is set when the product is first inserted. The CreatedBy is given a fixed value (System) to indicate who created the record. This is useful in scenarios where products are bulk-inserted by an automated process.

Customer Entity

Create a class file named **Customer.cs** within the Entities folder, and then copy and paste the following code.

```

using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Customer
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int CustomerId { get; set; } // Primary and Identity Column
        public string Name { get; set; }
        public string Email { get; set; }
        public string PhoneNumber { get; set; }
    }
}

```

```

        public List<Order> Orders { get; set; }
    }
}

```

Explanation of Customer Entity

CustomerId (DatabaseGeneratedOption.Identity):

The CustomerId is an auto-incremented primary key managed by the database. Using DatabaseGeneratedOption.Identity allows the database to generate a unique identifier for each new customer automatically. When using integer-based primary keys, DatabaseGeneratedOption.Identity is a simple and efficient way to automatically generate unique values, minimizing duplication risk.

Orders:

This property represents the list of orders associated with the customer. It allows us to navigate the relationship between the customer and their orders.

Order Entity

Create a class file named **Order.cs** within the Entities folder and then copy and paste the following code. The Order entity represents customer orders, with auto-generated OrderDate and status tracking.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Order
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int OrderId { get; set; } // Identity Primary Key
        public decimal TotalAmount { get; set; }
        public DateTime OrderDate { get; set; } // Default current date
        public string Status { get; set; } // Pending, Failed, Successful,
Default is Pending
        public int CustomerId { get; set; } // FK to Customer.SerialNumber
        public Customer Customer { get; set; } // Navigation Property
        public List<OrderItem> OrderItems { get; set; }
    }
}

```

Explanation of Order Entity

OrderId (DatabaseGeneratedOption.Identity):

The OrderId is an auto-incremented primary key generated by the database. It ensures a unique key for each order without requiring manual management.

OrderDate (DatabaseGeneratedOption.Computed):

When a new order is created, the OrderDate column is assigned a default value of the current date (GETDATE()). This helps automatically capture the date of order placement. The database generates this column, ensuring the current timestamp is accurate and set during the insertion.

Status (Default Value – Pending):

The Status field defaults to 'Pending'. This indicates the order's initial state before it's processed. Using a default value ensures consistency across all new records, reducing the need for explicit assignment by the application.

OrderItem Entity

Create a class file named **OrderItem.cs** within the Entities folder, and then copy and paste the following code. The OrderItem entity represents individual items in an order, with computed TotalPrice.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class OrderItem
    {
        public int OrderItemId { get; set; } // Identity Primary Key
        public int OrderId { get; set; } // FK to Order.OrderId
        public Order Order { get; set; } // Navigation Property
        public Guid ProductId { get; set; } // FK to Product.ProductId
        public Product Product { get; set; } // Navigation Property
        public int Quantity { get; set; }
        [Column(TypeName = "decimal(10,2)")]
        public decimal Price { get; set; }
        [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
        public decimal TotalPrice { get; set; } // Computed as Quantity *
        Price
    }
}
```

Explanation of OrderItem Entity

OrderItemId (DatabaseGeneratedOption.Identity):

The OrderItemId is an auto-incremented key used for each order item. Automatically managing the unique identifier for order items simplifies the insertion logic and ensures each item is easily distinguishable.

TotalPrice (DatabaseGeneratedOption.Computed):

The Total Price is computed as Quantity * Price. The database automatically calculates the value whenever an OrderItem is inserted or updated. This approach allows the calculation to be centralized in the database, ensuring consistency and reducing application-level computation.

Creating DbContext:

Create a class file named EFCoreDbContext.cs within the Entities folder, and then copy and paste the following code. This class provides configurations for entities and relationships. Here, we provide the necessary configurations for computed columns and default values using Fluent API.

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=ECommerceDB;Trusted_Connection=True;TrustSe
        }
        //Overriding the OnModelCreating method to configure the Default and (
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Configure dynamic default value for CreatedOn in Product entity
            modelBuilder.Entity<Product>()
                .Property(p => p.CreatedOn)
                .HasDefaultValueSql("GETDATE()");
            // Configure static or fixed default value for CreatedBy in Product entity
            modelBuilder.Entity<Product>()
                .Property(p => p.CreatedBy)
                .HasDefaultValueSql("'System'");
            // Configure dynamic default value for OrderDate in Order entity
            modelBuilder.Entity<Order>()
                .Property(o => o.OrderDate)
                .HasDefaultValueSql("GETDATE()");
            // Configure static or fixed default value for Status in Order entity
            modelBuilder.Entity<Order>()
                .Property(o => o.Status)
                .HasDefaultValue("Pending");
            // Configure computed column for CustomerId in Customer entity
            // CUST-SerialNumber
            modelBuilder.Entity<Product>()
                .Property(c => c.SKU)
                .HasComputedColumnSql("'PROD-' + CAST([Category] AS NVARCHAR)
CAST([SerialNumber] AS NVARCHAR)");
            // Configure computed column for TotalPrice in OrderItem entity
            modelBuilder.Entity<OrderItem>()
                .Property(oi => oi.TotalPrice)
                .HasComputedColumnSql("[Quantity] * [Price]");
        }
        // DbSet properties for each entity
        public DbSet<Product> Products { get; set; }
        public DbSet<Customer> Customers { get; set; }
        public DbSet<Order> Orders { get; set; }
        public DbSet<OrderItem> OrderItems { get; set; }
    }
}

```

Program Class:

Next, modify the Program class as follows. Here, we are demonstrating seeding and fetching data to validate DatabaseGenerated behavior.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                // Ensure the database is fresh by deleting and recreating it
                context.Database.EnsureDeleted(); // Delete existing database
                context.Database.EnsureCreated(); // Create new database
                based on the model
                // Seed Data
                Console.WriteLine("Seeding data into the database...");
                // Create a new product
                var product = new Product
                {
                    ProductId = Guid.NewGuid(), // PK without Identity; must
                    provide a unique value
                    Name = "Laptop",
                    Category = "ELECT",
                    Price = 1500.00m,
                    Quantity = 10,
                };
                context.Products.Add(product);
                // Create a new customer
                var customer = new Customer
                {
                    // CustomerId is a computed column
                    // SerialNumber is Identity and will be auto-generated
                    Name = "Pranaya Rout",
                    Email = "pranaya.rout@example.com",
                    PhoneNumber = "123-456-7890"
                };
                context.Customers.Add(customer);
                // Save changes to generate identity and computed fields for
                Product and Customer
                context.SaveChanges();
                // Create a new order
                var order = new Order
                {
                    // OrderId is Identity and will be auto-generated
                    CustomerId = customer.CustomerId,
                    TotalAmount = 1500.00m
                    // OrderDate has Default value as current date
                    // Status has a default value of Pending
                };
            }
        }
    }
}
```



```

context.Orders.Add(order);
// Save changes to generate identity and computed fields for
Order

context.SaveChanges();
// Create a new order item
var orderItem = new OrderItem
{
    // OrderItemId is Identity and will be auto-generated
    OrderId = order.OrderId, // FK to Order.OrderId
    ProductId = product.ProductId, // FK to Product.ProductId
    Quantity = 1,
    Price = 1500.00m
    // TotalPrice is a computed column based on Quantity *
Price

};
context.OrderItems.Add(orderItem);
// Save changes to generate identity and computed fields for
OrderItem

context.SaveChanges();
// Display the data
Console.WriteLine("\nData saved successfully. Displaying the
data:\n");
// Use the entities we have in memory, which have been
updated with database-generated values
Console.WriteLine($"Product:");
Console.WriteLine($"    \tProductId: {product.ProductId}, Name:
{product.Name}, SerialNumber: {product.SerialNumber}");
Console.WriteLine($"    \tSKU: {product.SKU}, CreatedOn:
{product.CreatedOn}, CreatedBy: {product.CreatedBy}");
Console.WriteLine($"    \nCustomer:");
Console.WriteLine($"    \tCustomerId: {customer.CustomerId},
Name: {customer.Name}");
Console.WriteLine($"    \tEmail: {customer.Email}, PhoneNumber:
{customer.PhoneNumber}");
Console.WriteLine($"    \nOrder:");
Console.WriteLine($"    \tOrderId: {order.OrderId}, TotalAmount:
{order.TotalAmount}, OrderDate: {order.OrderDate}");
Console.WriteLine($"    \tStatus: {order.Status}, CustomerId:
{order.CustomerId}");
Console.WriteLine($"    \nOrderItem:");
Console.WriteLine($"    \tOrderItemId: {orderItem.OrderItemId},
OrderId: {orderItem.OrderId}, ProductId: {orderItem.ProductId}");
Console.WriteLine($"    \tQuantity: {orderItem.Quantity}, Price:
{orderItem.Price}, TotalPrice: {orderItem.TotalPrice}");
}
}
}
}
}

```

Output:

```
Seeding data into the database...
```

```
Data saved successfully. Displaying the data:
```

```
Product:
```

```
  ProductId: 60778e3c-d10b-48ec-a6ad-cee3c8e91d6a, Name: Laptop, SerialNumber: 1  
  SKU: PROD-ELECT-1, CreatedOn: 17-11-2024 12:26:47, CreatedBy: System
```

```
Customer:
```

```
  CustomerId: 1, Name: Pranaya Rout  
  Email: pranaya.rout@example.com, PhoneNumber: 123-456-7890
```

```
Order:
```

```
  OrderId: 1, TotalAmount: 1500.00, OrderDate: 17-11-2024 12:26:47  
  Status: Pending, CustomerId: 1
```

```
OrderItem:
```

```
  OrderItemId: 1, OrderId: 1, ProductId: 60778e3c-d10b-48ec-a6ad-cee3c8e91d6a  
  Quantity: 1, Price: 1500.00, TotalPrice: 1500.00
```

Database Initialization:

- **EnsureDeleted():** Deletes the existing database. Please note that this is for demonstration purposes. Remove or comment in production to prevent data loss.
- **EnsureCreated():** Creates the database based on the defined model and configurations.

When Should We Use the DatabaseGenerated Attribute in Entity Framework Core?

The DatabaseGenerated attribute is used in scenarios where certain properties are managed by the database, not by the application code. The following are common scenarios for its usage:

- **Auto-Incrementing Primary Keys (DatabaseGeneratedOption.Identity):** Use DatabaseGeneratedOption.Identity when you want the database to generate unique values automatically for a property (like a primary key). When using integer-based primary keys (like int, bigint), the DatabaseGeneratedOption.Identity is commonly used to let the database auto-generate the primary key for each new row.
- **Computed Columns (DatabaseGeneratedOption.Computed):** Use DatabaseGeneratedOption.Computed when the value of a column is calculated by the database rather than supplied by the application. This is common for fields the database should automatically calculate (e.g., computed columns, timestamps, or fields that rely on database functions like GETDATE()).
- **Manual Value Insertion (DatabaseGeneratedOption.None):** Use DatabaseGeneratedOption.None when you don't want the database to generate the value automatically, and instead, the value is supplied by the application. When you want full control over a property and don't want the database to interfere with its value generation, we need to use DatabaseGeneratedOption.None.

In the next article, I will discuss the [TimeStamp Attribute in Entity Framework Core](#) with Examples. In this article, I try to explain the **DatabaseGenerated Data Annotation Attribute in Entity Framework Core** with Examples. I hope you enjoyed this article on DatabaseGenerated Attribute in EF Core with Examples.